

JAVA 17 OCP · YDS STUDY SERIES

Unit 08 · Lambdas and Functional Interfaces — Practice Quiz

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu belge Java 17/OCP odağında hazırlanmış **özgün çalışma soruları** içerir; gerçek sınav sorusu değildir. Önerilen süre 15–20 dakikadır. Her lambda için önce target type ve SAM signature'ını yazman önerilir.

Sorular

Soru 1

Aşağıdaki kodun çıktısı nedir?

```
import java.util.function.Predicate;

public class PredicateQuiz {
    public static void main(String[] args) {
        Predicate<Integer> positive = x -> {
            System.out.print("P");
            return x > 0;
        };
        Predicate<Integer> even = x -> {
            System.out.print("E");
            return x % 2 == 0;
        };

        System.out.print(positive.and(even).test(-2));
    }
}
```

- A. PEfalse
- B. Pfalse
- C. EPfalse
- D. Kod derlenmez.

Soru 2

Aşağıdaki program için doğru sonuç hangisidir?

```
import java.util.function.Predicate;

public class CaptureQuiz {
    public static void main(String[] args) {
        int limit = 3;
        Predicate<Integer> large = value -> value > limit;
        limit++;
        System.out.print(large.test(5));
    }
}
```

- A. true yazdırır.
- B. false yazdırır.
- C. Kod derlenmez; `limit` effectively final değildir.
- D. Kod derlenir fakat lambda çağrısında `IllegalStateException` fırlatır.

Soru 3

Geçerli method reference assignment'ları olan **iki seçeneği** seçin.

- A. `Supplier<String> a = String::new;`
- B. `Function<String, Integer> b = String::length;`
- C. `Predicate<String> c = String::toUpperCase;`
- D. `BiFunction<String, String, Integer> d = String::concat;`

Gerekli type'ların `java.util.function` paketinden import edildiğini varsayın.

Soru 4

Aşağıdaki interface için doğru ifade hangisidir?

```
interface TextRule {  
    int apply(String text);  
    boolean equals(Object other);  
}
```

- A. İki abstract declaration bulunduğu için functional interface değildir.
- B. `equals(Object)`, `Object`ın public method signature'ıyla eşleştiğinden SAM hesabına katılmaz; interface functional'dır.
- C. Yalnız `@FunctionalInterface` eklenirse functional olur.
- D. Return type `boolean` olmadığı için lambda target type olamaz.

Soru 5

Aşağıdaki tanımlar için doğru sonuç çifti hangisidir? `java.util.function.Function` import edilmiş ve satırlar bir method body içinde kabul edilmiştir.

```
Function<Integer, Integer> plusOne = x -> x + 1;
Function<Integer, Integer> timesTwo = x -> x * 2;

int first = plusOne.andThen(timesTwo).apply(3);
int second = plusOne.compose(timesTwo).apply(3);
```

- A. `first == 7, second == 8`
- B. `first == 8, second == 7`
- C. Her ikisi de 8
- D. Kod derlenmez; aynı input type'a sahip function'lar zincirlenemez.

Soru 6

Şu cümlelerin doğal Türkçe çevirisi ve doğru çıkarımı hangisidir?

A lambda may capture a local variable only if it is final or effectively final; the variable need not be explicitly declared final.

- A. “Bir lambda local variable'ı yalnız `final` veya effectively final ise capture edebilir; variable'ın açıkça `final` ilan edilmesi gerekmez.” Atamadan sonra değeri değişmeyen local variable da uygundur.
- B. “Lambda'nın kullandığı her local variable'a mutlaka `final` keyword'ü yazılmalıdır.” Effectively final yeterli değildir.
- C. “Lambda yalnız instance field capture edebilir.” Local variable erişimi yasaktır.
- D. “Capture edilen local variable lambda çalışırken serbestçe değiştirilebilir.” Compile-time kısıt yoktur.

Cevaplar ve açıklamalar

Soru 1 — B

- **A yanlış:** `Predicate.and()` short-circuit uygular; ilk predicate `false` olunca `even` çalışmaz ve `E` yazılmaz.
- **B doğru:** `positive` önce `P` yazdırıp `false` döndürür; dıştaki `print()` sonucu ekleyerek `Pfalse` üretir.
- **C yanlış:** `and()` soldaki predicate'i önce değerlendirir.
- **D yanlış:** İki lambda da `Predicate<Integer>` SAM signature'ına uyar.

Soru 2 — C

- **A ve B yanlış:** Program runtime aşamasına ulaşmaz.
- **C doğru:** Lambda `limit`i capture ettikten sonra `limit++` yeniden atama sayılır. Bu nedenle `variable effectively final` değildir ve kod **Does not compile**.
- **D yanlış:** Capture kuralı runtime exception değil compile-time kısıtlamadır.

Soru 3 — A ve B

- **A doğru:** No-arg `String` constructor'ı `Supplier<String>.get()` ile uyumludur.
- **B doğru:** Unbound instance method reference'ta `String` parameter receiver olur; `length()` bir `int` döndürür ve `Integer`a box edilir.
- **C yanlış:** `Predicate.test()` `boolean` bekler, `toUpperCase()` ise `String` döndürür.
- **D yanlış:** `BiFunction.apply()` burada `Integer` döndürmeliyken `concat()` `String` döndürür.

Soru 4 — B

- **A yanlış:** `Object`ın public method'larıyla eşleşen declaration'lar SAM sayımına yeni abstract contract olarak eklenmez.
- **B doğru:** Tek ilgili abstract method `apply(String)` dir.
- **C yanlış:** Annotation doğrulama sağlar; interface'i functional yapan şey tek abstract method contract'ıdır.
- **D yanlış:** Functional interface'in SAM'i herhangi bir geçerli return type'a sahip olabilir.

Soru 5 — B

- **A yanlış:** `andThen`, önce soldaki `plusOne` çalıştırır.
- **B doğru:** `andThen: (3 + 1) * 2 = 8; compose: (3 * 2) + 1 = 7.`
- **C yanlış:** Operation sırası farklı olduğu için ikinci sonuç `7` dir.
- **D yanlış:** İlk function'ın return type'ı ikinci function'ın input type'ıyla uyumlu olduğundan zincir geçerlidir.

Soru 6 — A

- **A doğru:** “`need not`”, “gerekmez” anlamı verir. Effectively final, variable'a ilk atamadan sonra yeniden değer atanmaması demektir.
- **B yanlış:** Explicit `final` keyword zorunlu değildir.
- **C yanlış:** Lambda local variable capture edebilir; koşul final/effectively final olmasıdır.
- **D yanlış:** Sonraki yeniden atama capture'ı compile-time'da geçersiz kılar.